

SIXAT SA de CV

Sistema de gestión de flotilla de taxis

Informe de cierre de proyecto

Autor	Andrés Herrera Garza
Materia	Ingeniería de Software — Proyecto final (Reto)
Institución	Universidad Tecmilenio
Repositorio	github.com (Taxis_Dashboard)
Sistema en línea	taxis-dashboard.onrender.com
Fecha del informe	26 de septiembre de 2026

1. Resumen ejecutivo

SIXAT es un dashboard web para la administración de una flotilla de taxis (choferes, vehículos, documentación y asignaciones), desarrollado como proyecto final individual de Ingeniería de Software. El sistema quedó funcionando en producción, con autenticación JWT basada en roles, un backend Flask/PostgreSQL desplegado en Render y un frontend estático publicado en GitHub Pages. Se implementó una suite de 53 pruebas unitarias con 83.67% de cobertura y un pipeline de CI/CD en GitHub Actions con tres etapas de verificación (pruebas, seguridad y calidad de código) antes de cualquier despliegue.

6 TABLAS / ENTIDADES	53 PRUEBAS UNITARIAS	83.7% COBERTURA DE PRUEBAS	9.05/10 CALIFICACIÓN PYLINT	0 HALLAZGOS BANDIT
-------------------------	-------------------------	-------------------------------	--------------------------------	-----------------------

2. Comparación: planeación vs. implementación

2.1 Alcance funcional

Módulo planeado (acta de constitución)	Estado final	Notas
Registro y gestión de choferes (datos personales, dirección, estatus)	Entregado	CRUD completo, con datos forzados a mayúsculas excepto nombre/apellidos (Title Case) y edición prellenada.
Gestión documental de choferes (foto, licencia, comprobante, contrato)	Entregado	Carga real de archivos (no solo texto): almacenamiento en disco del servidor, nombres únicos por UUID y descarga vía endpoint protegido.
Registro y edición de carros (información y estado operativo)	Entregado	CRUD completo; el <code>id</code> del carro es asignado por el usuario (no autoincremental) y se bloquea al editar.
Registro de citas de gobierno y mantenimientos	Parcial	El backend (API <code>/citas_gob</code> , CRUD completo y protegido por rol) se construyó desde una etapa temprana, pero no se llegó a construir la pantalla de interfaz para administrarlo desde el navegador. Es la brecha de alcance más relevante del proyecto y encabeza el plan de mejora (sección 4).
Consulta de asignación actual (chofer ↔ carro)	Entregado	Se integró directamente al flujo del dashboard: al seleccionar un carro se muestra su chofer asignado, con acceso directo a editarlo.
Historial de asignaciones	Entregado	Tabla <code>asignaciones</code> con <code>fecha_inicio</code> / <code>fecha_fin</code> , más una pantalla dedicada (<code>asignar.html</code>) para crear nuevas asignaciones respetando la regla de negocio de un carro/chofer activo a la vez.

Fuera del alcance original, agregado en el camino: hospedaje público del sistema (Render + GitHub Pages), autenticación con JWT y roles, y el pipeline de CI/CD con pruebas, seguridad y calidad de código — ninguno de estos tres estaba en el acta de constitución original, pero se volvieron requisito por la rúbrica final de la materia.

2.2 Requisitos no funcionales

Requisito planeado	Estado	Evidencia / observación
Seguridad: autenticación usuario/contraseña, hash de contraseñas	Cumplido, y ampliado	Se implementó JWT con expiración (8h) y control de acceso por rol (administrador/chofer) además del hash con <code>werkzeug.security (scrypt)</code> — más robusto que lo planeado originalmente.
Rendimiento: respuesta menor a 2 segundos	No verificado formalmente	No se midió con una herramienta de carga/benchmark. En uso normal la API responde de forma inmediata, salvo el "cold start" de ~30-50s del plan gratuito de Render tras inactividad (ver sección 3).
Disponibilidad: 24/7	Cumplido con matiz	El plan gratuito de Render "duerme" el servicio tras inactividad, generando un retraso en la primera petición. Funcionalmente disponible 24/7, pero no con tiempo de respuesta constante.
Usabilidad: interfaz intuitiva, capacitación mínima	Cumplido	Navegación consistente (barra superior en todas las páginas), mensajes de confirmación/error como toasts en vez de popups del navegador, y botones reales en vez de hipervínculos para acciones destructivas.

2.3 Metodología y arquitectura

Se siguió **Kanban** de forma informal (sin tablero visual documentado), tal como se planeó, dado que es un proyecto de una sola persona. La arquitectura se mantuvo **monolítica** como se decidió en el acta —un backend Flask único, sin microservicios—

aunque el despliegue final terminó dividido en dos servicios independientes (API en Render, sitio estático en GitHub Pages) por la naturaleza del hospedaje elegido; esto no contradice la decisión arquitectónica original, solo describe cómo se distribuyó en la nube.

2.4 Cronograma planeado vs. real

El cronograma se reconstruyó a partir del historial real de commits de Git (fechas exactas, no estimadas):

Etapas	Fecha planeada (Gantt, cronograma "actualizado al 28 ago")	Fecha real (evidencia en Git)	Desviación
Requerimientos y análisis	16–19 ago	13–17 ago (estructura, roadmap)	A tiempo
Diseño de base de datos (ER)	19–22 ago	24 ago (diagrama ER entregado)	~2 días de retraso
Backend: endpoints iniciales (GET)	28 ago	25 ago (pruebas de conexión y GET de carros/choferes)	Adelantado
Entrega de avance de clase	28 ago	28 ago (acta, documentos actualizados)	Exacto
Desarrollo backend (CRUD completo, 6 tablas)	3–15 sep	9–10 sep	Adelantado
Desarrollo frontend	12–21 sep	10 sep (estructura base), pausa , retomado 24–26 sep	Interrumpido y comprimido al final
Pruebas y deployment	21–24 sep	25–26 sep (deploy, JWT, pruebas y CI/CD)	~2 días de retraso, resuelto a tiempo
Entrega final	~27 sep	26 sep	A tiempo

Lo que realmente pasó: entre el 10 y el 24 de septiembre hubo un vacío de casi dos semanas sin commits (coincide con carga de otras materias y del trabajo en Deco Digital), seguido de tres días muy intensos (24–26 sep) donde se concentró casi todo el frontend, la autenticación, las pruebas y el CI/CD. El riesgo #1 del acta de constitución ("retraso por ser el único responsable del proyecto y tener otras materias") **se materializó tal cual se anticipó**, y la mitigación planeada en ese momento —invertir horas extra para recuperar el atraso— fue exactamente la que se usó para cerrar a tiempo.

2.5 Matriz de riesgos: lo planeado vs. lo que ocurrió

#	Riesgo	¿Se materializó?	Qué pasó realmente
1	Retraso por ser el único responsable y tener otras materias	Sí	Ver sección 2.4. Se resolvió con horas extra en la recta final, tal como estaba previsto en la contingencia.
2	Problemas técnicos de instalación/configuración de PostgreSQL	Sí	Ocurrió al migrar la base de datos a Render: errores de resolución de host (interno vs. externo) y de formato de backup (Plain vs. Custom) en pgAdmin. Se resolvió consultando documentación y con apoyo de IA, exactamente como marcaba la contingencia.
3	Cambios en el diseño de la base de datos a medio desarrollo	No, de forma relevante	El esquema se mantuvo estable; el único caso especial fue que el id de carros es asignado por el usuario en vez de autoincremental, decisión tomada desde el diseño y no un cambio a medio camino.
4	Filtración o acceso no autorizado a documentos sensibles	Riesgo real detectado a tiempo	Al hacer público el sitio se detectó que no existía autenticación real . Se corrigió antes de que representara un incidente, implementando JWT + roles + hash de contraseñas — validando que la mitigación planeada (autenticación con hash, acceso restringido) era la correcta.
5	Pérdida de código o avances por falla local o del equipo	No ocurrió	Commits frecuentes a GitHub evitaron cualquier pérdida, incluida una vez que se cometió por error un backup de la base de datos (16 KB) en un commit: se detectó y se removió en un commit posterior sin perder historial.

3. Lecciones aprendidas

3.1 Gestión de tiempo y alcance

- El riesgo de retraso por trabajar en solitario, identificado desde el acta de constitución, se cumplió casi al pie de la letra. Anticiparlo con una contingencia clara (horas extra en vez de improvisar) permitió cerrar a tiempo sin sacrificar calidad.
- Construir el backend completo antes que el frontend (como marcaba el plan) funcionó bien: cuando se retomó el frontend después de la pausa de dos semanas, la API ya estaba probada y estable, lo que aceleró mucho esa etapa.
- Los requisitos no funcionales de seguridad terminaron siendo más exigentes de lo previsto: pasar de "usuario y contraseña" (acta original) a JWT + roles fue una decisión correcta, pero no estaba dimensionada en el cronograma inicial.

3.2 Base de datos y backend

- Migrar de hábitos de Oracle a PostgreSQL requirió corrección constante: tipos (`NUMBER` → tipo entero/numérico de Postgres), funciones de fecha (`SYSDATE` → `NOW()`), y usar `UPDATE` para datos en vez de `ALTER TABLE`.
- Diseñar `documentos` como tabla flexible (una fila por documento) en vez de columnas fijas por tipo de documento fue clave para poder agregar la carga real de archivos sin rediseñar el esquema.
- La tabla de unión `asignaciones` con `fecha_inicio` / `fecha_fin` resultó ser la decisión de diseño más rentable del proyecto: permitió construir tanto la asignación actual como el historial completo con la misma estructura de datos.

3.3 Seguridad y autenticación

- Un sistema puede "funcionar" perfectamente y seguir siendo inseguro: la ausencia de autenticación no se hizo evidente hasta que el sistema se volvió públicamente accesible. La lección concreta: la autenticación debe diseñarse desde el principio si el sistema va a ser público, no agregarse al final.
- El hash de contraseñas (scrypt vía Werkzeug) es unidireccional por diseño: ni siquiera el desarrollador puede "ver" la contraseña real de un usuario, solo verificarla reaplicando el mismo algoritmo. Entender esto evitó intentar depurar el login revisando contraseñas en la base de datos.
- Un bug sutil (`login.js` llamaba a una función de `auth.js` , que deliberadamente no se carga en la página de login para evitar un bucle de redirección) enmascaró un login que en realidad ya funcionaba, y se leyó erróneamente como un problema del backend. Lección: al depurar, aislar primero si el error es de frontend o backend antes de sospechar de la lógica de negocio.

3.4 Control de versiones y despliegue

- `git add .` solo afecta desde el directorio actual hacia abajo — un error recurrente al trabajar desde subcarpetas.
- GitHub Pages solo permite `/ (root)` o `/docs` como carpeta de origen, y activar un dominio personalizado genera automáticamente un archivo `CNAME` que puede romper el sitio si el dominio no es real o no se posee (ocurrió con `www.sixat.com` , un dominio que no existe).
- En Render, el hostname interno de PostgreSQL solo es accesible entre servicios de Render; para conectarse desde una herramienta externa (pgAdmin, la propia laptop) hay que usar el hostname externo.
- Los backups de pgAdmin deben hacerse en formato **Custom**, no **Plain**, para poder restaurarse desde el diálogo de Restore; en formato Plain hay que ejecutarlo manualmente desde el Query Tool.

3.5 Pruebas, seguridad y CI/CD

- Aislar la base de datos con mocks (en vez de requerir una conexión real a PostgreSQL) fue lo que hizo posible correr 53 pruebas en segundos y, después, ejecutarlas automáticamente en GitHub Actions sin necesitar credenciales de base de datos en el pipeline.
- Un hallazgo de seguridad no siempre significa una vulnerabilidad real: Bandit marcó el `host="0.0.0.0"` del servidor como riesgo, pero es exactamente lo que Render requiere para exponer el servicio. Documentarlo con un comentario `nosec` es mejor práctica que ignorarlo o suprimirlo sin explicación.
- Automatizar pruebas, seguridad y calidad de código en un pipeline con etapas encadenadas (que el despliegue no corra si las pruebas o el análisis de seguridad fallan) da mucha más confianza que ejecutar estas herramientas manualmente antes de cada entrega.

3.6 Frontend y experiencia de usuario

- El navegador cachea el CSS y, de forma independiente, cualquier imagen referenciada dentro del CSS (`url()`) — cambiar el archivo no bastaba para ver el cambio; hubo que forzar la actualización con parámetros de versión (`?v=N`) en ambos lugares por separado.
- Reemplazar los `alert()` nativos del navegador por mensajes visuales propios (toasts) y los enlaces de "editar"/"eliminar" por botones reales mejoró notablemente la percepción de calidad del sistema con muy poco esfuerzo técnico.

4. Plan de mejora continua

Se organiza en tres horizontes. Cada acción incluye un criterio de medición concreto para poder verificar si se cumplió, y las marcadas como **Innovación** son propuestas que van más allá de completar el alcance original.

4.1 Corto plazo (siguiente iteración del sistema)

Acción	Criterio de medición	Prioridad
Construir la interfaz de citas de gobierno y mantenimientos (pantalla de listado/alta/edición sobre el endpoint <code>/citas_gob</code> ya existente)	Un administrador puede registrar, ver y editar una cita sin usar la API directamente; cierra la brecha de alcance de la sección 2.1	Alta
Reemplazar el almacenamiento de documentos en disco (efímero en el plan gratuito de Render) por un almacenamiento persistente (ej. un bucket S3-compatible o el disco persistente de pago de Render)	Un documento subido sigue disponible después de que el servicio se reinicie o redeploye; hoy no se garantiza	Alta
Crear un segundo usuario de prueba con <code>rol = 'chofer'</code> y verificar en producción que las rutas protegidas con <code>@requiere_admin</code> le devuelven 403	Prueba manual documentada (captura o log) mostrando la restricción funcionando en el sistema real, no solo en los tests automatizados	Media

4.2 Mediano plazo

Acción	Criterio de medición	Prioridad
Integración con la API de GPS de Protrack365 (<code>protrack365.com/api.html</code>) para obtener la ubicación en tiempo real de cada vehículo, aprovechando el sistema de rastreo ya instalado en la flotilla — explícitamente marcado como trabajo futuro desde el acta de constitución original	El dashboard muestra la última posición conocida de cada carro con un timestamp de actualización menor a 5 minutos, obtenida directamente de la API de Protrack365	Alta
Agregar búsqueda y paginación en los listados de choferes y carros	Los listados responden en menos de 2 segundos (el requisito no funcional original) incluso con más de 200 registros	Media
Medir formalmente el requisito de rendimiento (<2s) con una herramienta de carga (ej. Locust o k6) sobre los endpoints más usados	Reporte de carga documentado con tiempos de respuesta p50/p95 para los endpoints de choferes, carros y login	Media
Ampliar la cobertura de pruebas del 83.7% actual a un mínimo de 90%, cubriendo las líneas actualmente sin probar (manejo de errores y validaciones en <code>app.py</code>)	Reporte de <code>pytest --cov</code> mostrando $\geq 90\%$ en la corrida de CI/CD	Baja

4.3 Innovación

Propuesta	Criterio de medición
Mapa en vivo de la flotilla: una vez integrada la API de GPS (sección 4.2), mostrar todos los carros activos sobre un mapa interactivo en el dashboard, coloreados por estado (activo, fuera de servicio, baja), en vez de solo una lista de coordenadas	Un administrador puede ubicar visualmente cualquier carro activo en menos de 10 segundos desde que entra al dashboard
Notificaciones automáticas de citas de gobierno próximas a vencer (verificaciones, refrendos): un job programado que revise <code>citas_gob</code> y envíe un correo o notificación cuando falten N días para una cita, apoyándose directamente en la justificación original del proyecto ("asegurar que las citas de gobierno y mantenimiento se atiendan a tiempo")	Al menos una notificación real enviada y confirmada en un entorno de prueba con una cita configurada a 3 días de vencer
Panel de indicadores (KPIs) para el CEO/administrador: número de carros activos vs. fuera de servicio, choferes activos vs. reportados, y próximas citas de gobierno — respondiendo directamente a los stakeholders de "poder alto / interés alto" identificados en la matriz de stakeholders original	El panel se actualiza con datos reales de la base de datos (no estáticos) y carga en menos de 2 segundos

El hilo conductor de este plan de mejora es cerrar exactamente lo que el propio proyecto se propuso desde el acta de constitución y dejó como trabajo futuro (citas de gobierno con interfaz, GPS/Protrack365), y a partir de ahí construir las capacidades que ese alcance original hace posibles (mapa en vivo, notificaciones, panel de indicadores).